

Contents

Git & GitHub — kurz erklärt	1
Warum überhaupt?	1
Die vier wichtigsten Begriffe	1
Die vier Begriffe im Zusammenspiel	2
Warum wir das im Praktikum (noch) nicht so nutzen	2
Wenn ihr selbst ausprobieren wollt	2

Git & GitHub — kurz erklärt

Dieses Praktikum nutzt selbst **kein** Git (siehe Hinweis unten) — trotzdem lohnt sich ein kurzer Überblick, weil Git/GitHub euch in fast jedem Programmier-Kontext wieder begegnen werden. Diese Seite gibt nur die Grundidee, keine Detailtiefe.

Warum überhaupt?

Stellt euch vor, ihr schreibt zu zweit an derselben Datei. Ohne Werkzeug dafür: Ihr schickt euch die Datei ständig per USB-Stick oder E-Mail hin und her, überschreibt euch gegenseitig Änderungen, und irgendwann weiß keiner mehr, welche Version die aktuelle ist.

Git löst genau das Problem: Es speichert **jede Version** eures Codes, mit genauem Zeitpunkt und Begründung ("was habe ich geändert und warum"). Ihr könnt jederzeit zu einer älteren Version zurück, seht genau, wer was wann geändert hat, und könnt sogar gleichzeitig an verschiedenen Ideen arbeiten, ohne euch gegenseitig zu stören.

GitHub ist eine Webseite, die Git-Projekte online speichert und Teamarbeit zusätzlich erleichtert (z.B. Diskussionen zu Änderungen, Übersicht über alle Projekte, Backup in der Cloud). Man kann sich Git wie das Werkzeug und GitHub wie den öffentlichen Aufbewahrungsort dafür vorstellen.

Die vier wichtigsten Begriffe

1. Repository ("Repo")

Ein Repository ist einfach **ein Projektordner, den Git überwacht**. Git merkt sich darin die komplette Geschichte aller Änderungen — wie ein Ordner mit eingebauter Zeitmaschine.

2. Commit

Ein Commit ist ein **gespeicherter Schnappschuss** eures Codes zu einem bestimmten Zeitpunkt, zusammen mit einer kurzen Nachricht, die erklärt, was sich geändert hat.

Beispiel: Ihr habt gerade die Flucht-Logik in eurem Bot fertiggestellt. Dann macht ihr einen Commit mit der Nachricht "Bot flieht jetzt bei HP < 20". Später könnt ihr immer genau zu diesem Punkt zurückspringen, falls etwas kaputtgeht.

Faustregel: **kleine, oft gemachte Commits mit klarer Nachricht** sind besser als ein riesiger Commit am Ende des Tages mit der Nachricht "Änderungen".

3. Branch

Ein Branch ("Zweig") ist eine **eigene, abgezwigte Kopie** des Projekts, in der ihr etwas ausprobieren könnt, ohne die Hauptversion (meist `main` genannt) zu verändern.

Analogie: Stellt euch einen Baumstamm vor (`main` — die stabile, funktionierende Version). Ein Branch ist ein Ast, der vom Stamm abzweigt. Ihr könnt auf diesem Ast wild herumprobieren —

geht etwas schief, hat das keine Auswirkung auf den Stamm. Funktioniert die Idee, könnt ihr den Ast später wieder mit dem Stamm zusammenführen ("mergen").

Typischer Ablauf: Neuer Branch angriffslogik -> dort die Angriffslogik entwickeln und testen -> wenn sie funktioniert, zurück in main übernehmen.

4. Pull Request (PR)

Ein Pull Request ist eine **Anfrage**: "Ich habe in meinem Branch etwas fertig entwickelt — bitte schaut es euch an und übernehmt es in die Hauptversion (main)."

Das ist der Moment, an dem Teamkolleg:innen die Änderung vor der Übernahme gegenlesen können (ähnlich dem "Review (Pair-Check)" auf unserem Scrum-Board) — Fehler werden idealerweise entdeckt, bevor sie in der Hauptversion landen, nicht danach.

Die vier Begriffe im Zusammenspiel

Repository (Projektordner)

- └─ main-Branch (stabile Version)
 - └─ euer Branch "angriffslogik" (eigener Ast zum Ausprobieren)
 - └─ Commit 1: "Grundgerüst für Zielsuche"
 - └─ Commit 2: "Schießen bei Sichtlinie ergänzt"
 - └─ Pull Request -> Bitte in main übernehmen

Warum wir das im Praktikum (noch) nicht so nutzen

In diesem 3-Tage-Praktikum arbeitet jedes Team direkt in seinem eigenen Ordner (bots/teamX/), ohne Git — das hält den Fokus auf dem Kotlin-Lernen und vermeidet zusätzliche Werkzeug-Einarbeitung in kurzer Zeit. Die Integration am Turniertag macht der Dozent manuell. Das ist eine bewusste Vereinfachung für dieses Praktikum, kein Zeichen dafür, dass Git nicht wichtig wäre — in echten Software-Projekten (auch kleinen) ist es praktisch immer im Einsatz.

Wenn ihr selbst ausprobieren wollt

Ein guter, kostenloser nächster Schritt außerhalb des Praktikums: github.com einen Account anlegen und ein eigenes kleines Projekt hochladen. GitHub selbst bietet dafür eine kurze interaktive Einführung ("Hello World"-Guide), die genau diese vier Begriffe an einem echten Mini-Beispiel durchspielen lässt.